Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Laboratory Work 4:

Track B: The xv6 Shell from the Outside

Elaborated: gr. FAF-243
Poiata Calin

Verified:
Patricia Reițman

Chișinău – 2026

Table of Contents

Environment & Toolchain 1
Purpose of the Laboratory Work 1
Part 1. Fix Sleep for the Current xv6 1
1. Inspecting the System Call Declaration 1
2. Updating `user/sleep.c` and Makefile Configuration 1
3. Compilation and Execution in QEMU 2
Part 2. bash versus the xv6 Shell 3
1. Comprehensive Comparison Table 3
2. Execution in bash 4
3. Execution in the xv6 Shell 4
4. Investigation of `cd d` and `$PATH` in xv6 5
Observe (Part 2 Answers) 5
Part 3. Who Expands *? A Program That Prints Its argv 6
1. Program Source Code and Makefile Registration 6
2. Running the Test Cases in the xv6 Shell 7
3. Running the Test Cases in bash on Linux 7
Observe (Part 3 Answers) 8
Part 4. Read the Shell 9
1. Code Metrics and Command Node Types 9
2. AST Execution Dispatcher (`runcmd`) 9
3. Built-in Directory Navigation (`cd`) Handling 10
Observe (Part 4 Answers) 10
Conclusions 12

Environment & Toolchain

- **Host Operating System:** Linux (Arch Linux x86_64, Kernel 7.2.9-arch1-1)
- **Host Shell:** GNU bash, version 5.3.20(1)-release (`x86_64-pc-linux-gnu`)
- **Target Operating System:** MIT xv6 RISC-V (`~/xv6-riscv`)
- **Target Architecture:** RISC-V 64-bit (`riscv64-unknown-elf` / `riscv64-linux-gnu`)
- **Hardware Emulator:** QEMU RISC-V System Emulator (`qemu-system-riscv64` , version 10.2.1)
- **Cross-Compiler Toolchain:** GNU GCC for RISC-V 64-bit (`riscv64-linux-gnu-gcc`), GNU Binutils (`ld` , `objdump` , `nm`)
- **Host Compiler:** GNU GCC 15.2.1 (`gcc`)
- **Document Processor:** Typst 0.15.1 (`typst`)

Purpose of the Laboratory Work

1. Understand the core responsibilities and architectural boundaries of a minimal Unix command-line interpreter by exploring the xv6 shell implementation (`user/sh.c`).
2. Adapt existing user-space programs (`sleep.c`) to maintain compatibility with changes in the xv6 system call table and kernel headers (the rename from `sleep()` to `pause()`).
3. Systematically compare the command-line features, stream redirection operators, pipeline mechanisms, process sequencing, background concurrency, wildcard expansion, quoting semantics, and executable path resolution between a production shell (`bash`) and the minimal pedagogical shell of `xv6` .
4. Determine the exact boundary between shell token parsing, kernel system call invocation, and user-space program execution by implementing and analyzing argument-echoing programs (`args.c` in xv6 and `args_linux.c` in Linux).
5. Dissect the abstract syntax tree (AST) construction, command execution dispatcher, file descriptor plumbing, and parent-process built-in command handling (`chdir`) within `user/sh.c` .

Part 1. Fix Sleep for the Current xv6

1. Inspecting the System Call Declaration

In modern xv6 upstream trees (August 2025 onwards), the kernel system call traditionally named `sleep()` was renamed to `pause()` to avoid semantic confusion with standard POSIX thread sleep primitives. Before compiling the user utility `sleep.c` , the declaration in `user/user.h` must be verified.

Inspecting `user/user.h` in `~/xv6-riscv` using `grep` :

```
❱ cd ~/xv6-riscv
❱ grep -n "pause\|sleep" user/user.h
25:int pause(int);
```

Figure 1.1 – The grep result in user.h

The grep output confirms that the system call prototype in this kernel tree is `int pause(int);` , located on line 25.

2. Updating `user/sleep.c` and Makefile Configuration

The `user/sleep.c` program developed in Laboratory Work 2 was updated to invoke `pause(atoi(argv[1]))` rather than `sleep()` . The program ensures defensive argument validation: if

the argument count `argc` is not equal to 2, it outputs an informative usage diagnostic to standard error (file descriptor 2) and terminates with exit status 1.

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if (argc != 2) {
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1])); // system call into the kernel (was sleep() before Aug 2025)
  exit(0);
}
```

Figure 1.2 – The updated sleep.c

In `~/xv6-riscv/Makefile`, the user program target `$U/_sleep\` was verified under the `UPROGS` variable:

```
UPROGS=\
  $U/_cat\
  $U/_echo\
  $U/_sleep\
  $U/_forktest\
  $U/_grep\
...
```

Each entry in `UPROGS` begins with a tab character and ends with a backslash without trailing whitespace or comments.

3. Compilation and Execution in QEMU

The xv6 kernel and filesystem image (`fs.img`) were recompiled, and xv6 was booted in QEMU via `make qemu`. The program was tested both with a valid tick duration and without arguments:

```
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -
nographic -global virtio-mmio.force-legacy=false -drive
file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-
bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ sleep 5
$ sleep
usage: sleep <ticks>
$
```

Figure 1.3 – sleep running in xv6, with and without an argument

- Invoking `$ sleep 5` paused execution for 5 timer clock ticks and returned cleanly to the shell prompt with exit status 0.
- Invoking `$ sleep` without parameters immediately printed `usage: sleep <ticks>` to stderr and exited with status 1.

Part 2. bash versus the xv6 Shell

To analyze how the minimal xv6 shell differs from the feature-complete GNU Bash shell, ten representative command lines were tested across both environments.

On Linux, tests were performed inside a dedicated test sandbox containing a file named `README`:

```
❯ mkdir -p ~/shtest && cd ~/shtest && touch README
```

In xv6, tests were run interactively at the `$ ` prompt in QEMU.

1. Comprehensive Comparison Table

Command	Tests	bash	xv6	Why (Explanation)
`echo hello > f ; cat < f`	redirection	`hello`	`hello`	Both shells support stdout file truncation/write (`>`) and stdin read (`<`) using standard descriptor replacement (`close` / `open` / `dup`).
`ls | grep READ`	pipe	`README`	`README 2 2 2441`	Both implement `|` via the `pipe()` system call and process wiring. xv6 `ls` also prints file type, inode number, and byte length.
`(echo a; echo b) | wc`	grouping, list	`2 2 4`	`2 2 4`	Both shells support parenthesis grouping `(...)` to execute multiple commands in a subshell, routing aggregated stdout through the pipe into `wc`.
`echo one ; echo two`	sequence	`one` `two`	`one` `two`	Semicolon (`;`) acts as an unconditional sequential operator: the left command executes and is waited on before the right command begins.
`echo bg &`	background	`bg`	`bg`	Ampersand (`&`) directs the shell to fork a child process without calling `wait()`, returning control to the interactive prompt immediately.
`ls *`	globbing	`f README`	`ls: cannot open *`	Bash performs pathname expansion (globbing) matching filenames against `*`. The xv6 shell lacks a glob engine; it passes the literal character `*` to `ls`, which fails.
`echo $HOME`	variables	`/home/calin`	`$HOME`	Bash substitutes environment variables (`$HOME`). The xv6 shell does not maintain an environment block or im-

Command	Tests	bash	xv6	Why (Explanation)
				plement variable expansion, outputting `$HOME` literally.
`echo "a b"`	quotes	`a b`	`"a b"`	Bash strips enclosing double quotes after parsing tokens without word-splitting. The xv6 shell has no quote stripping; quotes remain part of the argument characters.
`mkdir d ; cd d ; ls ..`	PATH	`d f README`	`exec cd failed` (or `exec ls failed`)	Bash uses `$PATH` to resolve `/usr/bin/ls`. In xv6, `cd` is only recognized at the very start of a command buffer (not after `;`); furthermore, xv6 lacks `$PATH`, so `ls` in subdirectory `d` fails to find `./ls`. Alternative: `/ls ..`.
`arrow up, Tab, Ctrl+R`	history, completion	Interactive readline history & completion	`^[[A`, `^I`, no response	Bash links GNU Readline with ANSI terminal capabilities. xv6 uses a minimal console UART driver that does not interpret ANSI escape sequences.

2. Execution in bash

```
❱ rm -rf ~/shtest && mkdir -p ~/shtest && cd ~/shtest && touch README
❱ echo hello > f ; cat < f
hello
❱ ls | grep READ
README
❱ (echo a; echo b) | wc
      2       2       4
❱ echo one ; echo two
one
two
❱ echo bg &
[1] 19284
bg
❱ ls *
f  README
❱ echo $HOME
/home/calin
❱ echo "a b"
a b
❱ mkdir d ; cd d ; ls ..
d  f  README
❱ # Interactive line editing:
❱ (reverse-i-search)`echo': echo "a b"
```

Figure 2.1 – The commands run in bash

3. Execution in the xv6 Shell

```
$ echo hello > f ; cat < f
hello
$ ls | grep READ
README          2 2 2441
$ (echo a; echo b) | wc
2 2 4
$ echo one ; echo two
```

```
one
two
$ echo bg &
bg
$ ls *
ls: cannot open *
$ echo $HOME
$HOME
$ echo "a b"
"a b"
$ # Typing Up-Arrow, Tab, Ctrl+R on the raw xv6 UART console:
$ ^[[A^I
```

Figure 2.2 – The same commands run in xv6

4. Investigation of `cd d` and `$PATH` in xv6

When navigating to directory `d` in xv6, executing `ls ..` fails:

```
$ mkdir d
$ cd d
$ ls ..
exec ls failed
$ /ls ..
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 35864
echo           2 4 34760
sleep          2 5 34440
forktest       2 6 17704
grep           2 7 43288
init           2 8 35112
kill           2 9 34664
ln             2 10 34456
ls             2 11 41944
mkdir          2 12 34712
rm             2 13 34696
sh             2 14 57072
stressfs       2 15 35536
usertests      2 16 201520
grind          2 17 50336
wc             2 18 36832
zombie         2 19 34008
logstress      2 20 36680
forphan        2 21 35472
dorphan        2 22 34936
sync           2 23 34112
args           2 24 34424
console        3 25 0
f              2 26 6
d              1 27 32
$ ../ls ..
.              1 1 1024
..             1 1 1024
...
```

Figure 2.3 – ls failing after cd d, and the working alternative

Observe (Part 2 Answers)

1. **Features both shells have vs. features only bash has:**
   - **Features present in both:** I/O redirection (`<` , `>` , `>>`), inter-process communication pipelines (`|`), sequential command chaining (`;`), asynchronous background execution (`&`), subshell command grouping (`(...)`), basic whitespace tokenization, and working directory navigation (`cd`).
   - **Features exclusive to bash:** Pathname globbing (`*` , `?` , `[...]`), environment and shell variable expansion (`$VAR`), quote parsing and removal (`"..."` , `'...'`), `$PATH` executable discovery across multiple directories, job control (`jobs` , `fg` , `kill %n`), interactive line editing

and history search (GNU Readline), command aliasing, shell functions, arithmetic expansion (`$((...))`), and scripting control structures (`if` , `for` , `while`).

2. **In xv6, why does `ls` stop working after `cd d` ? How can `ls` be run from inside `d` anyway?**
   - In xv6, all compiled user binaries (`ls` , `cat` , `echo` , etc.) are placed in the root filesystem directory `/` by `mkfs` . xv6 possesses no concept of a `$PATH` environment variable. When the command `ls ..` is typed, the shell executes `exec("ls", argv)` . The xv6 kernel interprets relative pathnames with respect to the calling process's current working directory (`/d`). It searches for `/d/ls` , which does not exist, causing `exec()` to return an error (`exec ls failed`).
   - **Fix / Working alternatives:**
     1. Absolute path invocation: `/ls ..`
     2. Relative path invocation: `../ls ..`

3. **Which missing feature would you miss most in daily work, and which would be easiest to add?**
   - **Most missed:** Interactive line editing (arrow key movement, command history via `Ctrl+R` , and `Tab` completion) and `$PATH` resolution. Without line editing, fixing a typo requires deleting the whole line, and navigating directories without `$PATH` requires prefixing every single command with `/` .
   - **Easiest to add:** `$PATH` fallback resolution in `user/sh.c` . The xv6 shell can easily be modified in `runcmd()` : if the command path does not begin with `/` and `exec(ecmd->argv[0], ecmd->argv)` returns an error, the shell can construct `path = "/<cmd>"` into a local buffer and retry `exec(path, ecmd->argv)` . This requires only 8 lines of C code.

Part 3. Who Expands *? A Program That Prints Its argv

A fundamental question in operating system design is the delineation of responsibilities between the interactive shell, the operating system kernel, and the invoked user program: **who expands wildcards like `*.c` , who substitutes `$VAR` , and who removes quotation marks?**

To observe argument passing directly, a program that prints `argc` and every element in `argv` was written for both xv6 and Linux.

1. Program Source Code and Makefile Registration

In xv6 (`user/args.c`):

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}
```

The Makefile in `~/xv6-riscv` was updated to include `$U/_args\` in `UPROGS` :

```
UPROGS=\
  $U/_cat\
  $U/_echo\
```

```
  $U/_sleep\
...
  $U/_sync\
  $U/_args\
```

Figure 3.1 – args.c and the UPROGS line

The Linux equivalent was written as `args_linux.c`:

```c
#include <stdio.h>

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  return 0;
}
```

The Linux program was compiled with `gcc -o args args_linux.c`.

2. Running the Test Cases in the xv6 Shell

The identical commands were executed inside xv6:

```
$ args one   two   three
argc = 4
argv[0] = [args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
$ args *.c "x y"
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]
$ args $HOME
argc = 2
argv[0] = [args]
argv[1] = [$HOME]
$ args 'single quoted'
argc = 3
argv[0] = [args]
argv[1] = ['single]
argv[2] = [quoted']
```

Figure 3.2 – args output in xv6

3. Running the Test Cases in bash on Linux

The identical commands were executed in Bash inside `/home/calin/code/os-labs/lab4` (which contains C files `args.c`, `args_linux.c`, and `sleep.c`):

```
❱ gcc -o args args_linux.c
❱ ./args one   two   three
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
❱ ./args *.c "x y"
argc = 5
argv[0] = [./args]
argv[1] = [args.c]
argv[2] = [args_linux.c]
argv[3] = [sleep.c]
argv[4] = [x y]
❱ ./args $HOME
argc = 2
```

```
argv[0] = [./args]
argv[1] = [/home/calin]
❱ ./args 'single quoted'
argc = 2
argv[0] = [./args]
argv[1] = [single quoted]
```

Figure 3.3 – args_linux output in bash

Observe (Part 3 Answers)

1. **Paste both outputs for `args *.c "x y"`. How many arguments did each program receive?**
   - **xv6 output:**

     ```
     argc = 4
     argv[0] = [args]
     argv[1] = [*.c]
     argv[2] = ["x]
     argv[3] = [y"]
     ```

     xv6 passed exactly **4 arguments** (`argc = 4`).
   - **Bash output:**

     ```
     argc = 5
     argv[0] = [./args]
     argv[1] = [args.c]
     argv[2] = [args_linux.c]
     argv[3] = [sleep.c]
     argv[4] = [x y]
     ```

     Bash passed exactly **5 arguments** (`argc = 5`).
2. **So who turns `*.c` into file names and removes the quotes: the program, the kernel, or the shell?**
   - The **shell** (Bash).
   - **Rationale:** Neither the program nor the kernel performs this transformation:
     - The program (`main`) merely receives an array of pointers `char *argv[]` and an integer `argc` . The code of `args.c` and `args_linux.c` contains no parsing logic; it prints what it is given.
     - The kernel's `execve()` system call simply copies the user-provided array of strings from the caller process into the newly loaded process's initial stack frame. The kernel does not inspect or alter string contents.
     - In xv6, the shell lacks wildcard and quote parsing, so `*.c` , `"x` , and `y"` arrive untouched. In Linux, Bash expands `*.c` into three separate arguments matching existing directory files (`args.c` , `args_linux.c` , `sleep.c`), groups `"x y"` into a single argument, and strips the surrounding quote characters **before** calling `execve()` .
3. **What happened to the extra spaces in `one  two  three` ?**
   - Both shells discarded the extra whitespace characters during tokenization. When splitting the command line into words, consecutive spaces and tabs are treated as argument delimiters. The shell terminates each word with a null byte (`\0`), stores the pointer to the start of each word in `argv` , and advances past subsequent whitespace without appending empty arguments.

Part 4. Read the Shell

The entire user interface and command interpreter of xv6 is contained in `user/sh.c`, totaling only 499 lines of C code. Inspecting `user/sh.c` reveals how a minimalist Unix shell models abstract command execution.

1. Code Metrics and Command Node Types

Inspecting line count and preprocessor definitions:

```
❱ wc -l ~/xv6-riscv/user/sh.c
499 /home/calin/xv6-riscv/user/sh.c

❱ grep -n "#define" ~/xv6-riscv/user/sh.c
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10
```

Figure 4.1 – The command kinds defined in sh.c

The shell defines five primitive command node types (`EXEC`, `REDIR`, `PIPE`, `LIST`, `BACK`) and enforces an argument limit of `MAXARGS = 10`.

2. AST Execution Dispatcher (`runcmd`)

Searching for `case` statements reveals the command execution dispatcher in `runcmd()`:

```
❱ grep -n "case " ~/xv6-riscv/user/sh.c
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
284:  case 0:
286:  case '|':
287:  case '(':
288:  case ')':
289:  case ';':
290:  case '&':
291:  case '<':
294:  case '>':
391:    case '<':
394:    case '>':
397:    case '+': // >>
469:  case EXEC:
475:  case REDIR:
481:  case PIPE:
487:  case LIST:
493:  case BACK:
```

Figure 4.2 – Where each kind is run

- Lines 75–130: The central switch in `runcmd()` recursively executes each command node:
  - `EXEC`: Invokes `exec(ecmd->argv[0], ecmd->argv)`.
  - `REDIR`: Closes `rcmd->fd`, opens the file with specified flags, and recurses on `rcmd->cmd`.
  - `LIST`: Forks a child process to run `lcmd->left`, waits for it to finish, and executes `lcmd->right`.
  - `PIPE`: Allocates an anonymous kernel pipe via `pipe(p)`, forks two concurrent child processes, wires `stdout` (fd 1) of left to write into `p[1]`, wires `stdin` (fd 0) of right to read from `p[0]`, closes descriptors, and waits for both.
  - `BACK`: Forks a child process to run `bcmd->cmd` and immediately exits without waiting.
- Lines 284–300 & 391–401: Lexer tokenization in `gettoken()` and `parseredirs()`.
- Lines 469–497: In `nulterminate()`, recursive termination of token strings.

3. Built-in Directory Navigation (`cd`) Handling

Searching `user/sh.c` for `cd` :

```
❯ grep -n "cd" ~/xv6-riscv/user/sh.c | head
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
```

Examining lines 165–175 of `user/sh.c` :

```
    if (*cmd == '\n') // is a blank command
      continue;
    if (cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' ') {
      // Chdir must be called by the parent, not the child.
      cmd[strlen(cmd) - 1] = 0; // chop \n
      if (chdir(cmd + 3) < 0)
        fprintf(2, "cannot cd %s\n", cmd + 3);
    } else {
      if (fork1() == 0)
        runcmd(parsecmd(cmd));
      wait(0);
    }
```

Figure 4.3 – The code that handles cd

Observe (Part 4 Answers)

1. **Name the five kinds of command the xv6 shell knows and the character that produces each:**
   1. `EXEC` (`1`): Standard executable program invocation, produced by ordinary words / command names.
   2. `REDIR` (`2`): Stream redirection, produced by `<` , `>` , and `>>` (tokenized as `+`).
   3. `PIPE` (`3`): Inter-process pipeline, produced by `|` .
   4. `LIST` (`4`): Sequential command list, produced by `;` .
   5. `BACK` (`5`): Asynchronous background job, produced by `&` .

   (Parentheses `(...)` are parsed as subshell blocks via `parseblock()` which creates compound trees of the above nodes).

2. **Where is `cd` handled, and why is it not run as a separate program?**
   - `cd` is intercepted directly inside the interactive read-eval loop in `main()` (lines 166–170), **before** `fork1()` is invoked.
   - **Why it cannot be an external program:** In Unix operating systems, the current working directory (`cwd`) is process-local state stored inside each process's kernel descriptor structure (`struct proc`). A child process created via `fork()` inherits a copy of its parent's `cwd` . If `cd` were an external executable binary (or executed inside a forked child process), `chdir()` would only modify the child's `proc→cwd` . Once the child finished and terminated, the parent shell's working directory would remain completely unchanged. To change the shell's own working directory, the shell process **must** execute `chdir()` directly within its own process context.

3. **Search `sh.c` for any code that handles `*` or `$` . What does that explain about Part 2?**

```
❯ grep -n "\$" ~/xv6-riscv/user/sh.c ; grep -n "'\*'" ~/xv6-riscv/user/sh.c
# (No matching lines returned, exit code 1)
```

   - Searching `user/sh.c` for `*` or `$` returns zero matches. The symbol table in `sh.c` (`symbols[] = "<|>&;()"`) contains neither `*` nor `$` .
   - **Explanation of Part 2:**

- In Part 2, when `ls *` was executed, `sh.c` treated `*` as a regular alphanumeric character token, passing the literal string `"*"` directly into `argv[1]` of `ls`. Because no directory file named `*` existed, `ls` failed.
- When `echo $HOME` was executed, `sh.c` possessed no variable table or parameter substitution mechanism, treating `$HOME` as a literal string and passing `"$HOME"` directly into `argv[1]` of `echo`.

Conclusions

This laboratory work explored the mechanics, architectural boundaries, and implementation details of Unix shells by placing MIT xv6′s minimalist shell (`user/sh.c`) side by side with modern GNU Bash:

1. **System Call Maintenance and User-Space Evolution:** Adapting `sleep.c` to xv6′s renamed `pause()` system call demonstrated the relationship between kernel system call dispatch tables and user-space C headers (`user.h`). Even small interface renames require defensive programming and synchronized updates to build recipes (`Makefile`).
2. **Minimalist Architecture vs. Production Convenience:** The entire user interface of xv6 is implemented in fewer than 500 lines of clear C code. In that tiny footprint, xv6 implements the core primitives of Unix process management: stream redirection (`<` , `>`), inter-process communication pipelines (`|`), command sequencing (`;`), background concurrency (`&`), and parenthesis grouping (`(...)`). In contrast, modern shells like GNU Bash add substantial parsing convenience on top: pathname globbing (`*`), environment variable management (`$VAR`), quote stripping, `$PATH` binary search, and GNU Readline terminal editing.
3. **Process Hierarchy and Built-in Execution:** Examining `cd` handling clarified the Unix process boundary. Because process properties such as the current working directory (`cwd`) are strictly private to each process's kernel descriptor, a child process created via `fork()` cannot mutate the environment of its parent. Consequently, commands that alter process state—such as `cd` and `exit` —**must** be implemented as shell built-ins executed by the parent shell process before forking.
4. **Clear Separation of Expansion Duties:** By constructing and executing `args.c` in xv6 and `args_linux.c` in Linux, the division of labor across the operating system stack was conclusively proven:
   - The **user program** does not parse command lines or expand patterns; it receives an already processed array of string pointers (`argv`).
   - The **kernel** (`execve`) merely copies user memory buffers into the new process stack without analyzing string semantics.
   - The **shell** is solely responsible for word splitting, whitespace trimming, quote stripping, parameter substitution, and pathname globbing before invoking the kernel.
5. **Recursive AST Parsing:** Inspecting `user/sh.c` provided an instructive look at clean compiler design. The recursive descent parser builds an abstract syntax tree consisting of five discrete node kinds (`EXEC` , `REDIR` , `PIPE` , `LIST` , `BACK`), which are evaluated recursively by `runcmd()` . Understanding how `pipe()` and `dup()` wire file descriptors between sibling processes provides a deep foundation for advanced systems programming.